

Mini Group Assignment – Operating Systems Objective

This mini project is designed to help students gain practical knowledge of operating system concepts using C programming. The primary focus is on process creation, program execution control, and basic inter-process communication simulation. Students are expected to develop a modular menu-driven system consisting of multiple executable programs.

Group Formation

- Each group must consist of **4 to 5 members**.
- Students are responsible for forming their own groups.
- If a group cannot find a full set of members, the instructor may assign remaining students to groups.

Project Overview

Students must design and implement a **menu-driven modular C program** that demonstrates operating system process management concepts.

The project contains two major subsystems:

1. **Fork-Based Task Executor System**
2. **Producer–Consumer Simulation System**

The program should follow modular programming principles where each subsystem is compiled as a separate executable file and invoked using process execution commands.

Main Menu Controller Program

The main program acts as the central controller of the project.

Menu Options

Students must display the following menu in the main controller program:



Select Project:

1. Fork-Based Task Executor System
2. Producer-Consumer Simulation System
0. Quit

Implementation Requirements

- Use the `fork()` system call to create a child process when a subsystem is selected.
- Use `exec()` family functions such as `execl()` or `execvp()` to execute subsystem programs.
- The parent process must use `wait()` to ensure that the child process completes execution before returning to the menu.
- Selecting **0 (Quit)** should terminate the program.

Fork-Based Task Executor System

This subsystem demonstrates process creation and execution of independent tasks.

Internal Menu

Fork Task Executor:

1. Factorial Calculation
2. Sorting Numbers
0. Quit

Factorial Program

- Prompt the user to enter an integer number.
- Calculate the factorial using either recursion or looping.
- Display the result on the screen.

Hint: Factorial of 5 should display as 120.

Important Note:

1. The program accepts **only integer input**.
2. String or character input is not supported.

Sorting Program

- Prompt the user to enter:
- Number of elements
- List of integer numbers

Perform sorting using a simple sorting algorithm such as bubble sort or similar.

Example:

- Input numbers: 5 6 7 1 3
- Output sorted array: 1 3 5 6 7

Hint:

- Use integer array storage.
- Sorting must be displayed after completion.

Producer–Consumer Simulation System

This module simulates communication between producer and consumer processes using a **single-slot shared buffer**.

Buffer Design

- Use POSIX shared memory mechanisms such as:
 - `shm_open()`
 - `mmap()`
 - `ftruncate()`

The buffer should be a **single integer storage slot**.

Important Rule:

- The producer program must **overwrite** the buffer value.
- Appending multiple values is not required.
- Multi-slot queues are not required.

Producer Module

- Accept only integer input from the user.
- If invalid input is provided, display an error message.
- Store the value in the shared memory buffer by overwriting the previous value.

Example:

```
Enter integer value to produce: 25
Produced value = 25
```

Consumer Module

- Read and display the latest value stored in shared memory.

Example:

```
Consumed value = 25
```

Note:

- Consumer reads only the most recently produced value.

Process Control Requirements

- Use `fork()` for process creation.
- Use `exec()` family functions to execute child programs.
- Parent process must wait using `wait()` before continuing execution.

Compilation and Execution

Students should submit the project in **three separate folders inside a ZIP file**:

1. **Source Code Folder**
 - Contains all `.c` source files.
2. **Compiled Executables Folder**
 - Contains all executable program files.
3. **Team Contribution Report Folder**
 - Contains documentation describing module-wise team member contributions.

Compilation Commands (Recommended)

```
gcc main_menu.c -o main_menu
gcc fork_tasks.c -o fork_tasks
gcc factorial.c -o factorial
```



```
gcc sorting.c -o sorting
gcc producer_consumer.c -o producer_consumer
gcc producer.c -o producer
gcc consumer.c -o consumer
```

Execution Command

./main_menu

Submission Guidelines

- Do not mix source code, executables, and report files in the same directory.
- Clearly document team member contributions.
- Programs that append data instead of overwriting the shared buffer may receive reduced marks.
- Proper modular design, process synchronization control, and code readability will be considered during evaluation.

Evaluation Expectations

Students should be able to explain:

- Process creation using fork()
- Program execution using exec family functions
- Parent-child process communication
- Shared memory simulation
- Modular programming structure